INDEX

NAME: Rakesh H STD.: ____ SEC.: ____ ROLL NO.: 415 SUB.: Computer Networks

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	20/07/26	Basic Linux Commands in Networking		
2.	27/07/26	Study of different types of Network Cables		
3.	03/08/26	Study of Packet tracer and overview of UI		
4.	04/08/26	Setup and Configuration of LAN using switch and ethernet cable		
5.	10/08/26	Packet Sniffer tool Wireshark		
6.	11/08/26	Error detection and correction.		
7.	17/08/26	Flow control - data link layer		
8.	18/08/26	NMAP to discover hosts using scans		
9.	25/08/26	Implementation of subnetting in cisco Packet tracer simulation		
10.	08/09/26	Internetworking with cisco Packet tracker		
11.	15/09/26	Routing at Network layer		
12.	22/09/26	End-to-End Communication at transport layer		
13.	28/09/26	Implementation of Ping program using Python		
14.	29/09/26	Implementation of Packet sniffing using raw sockets		
15.	06/10/26	Analyze various types of servers using webalizer tool		

Completed 1/10/26

EXP No: 1

20/07/26

Basic Linux Commands in Networking

Aim:

To study, execute, and understand the practical applications of fundamental networking and troubleshooting commands in a computer network.

Commands:

* arp -a

Displays the IP address of your computer along with the IP and MAC addresses of recently contacted devices stored in the ARP cache!

O/P

$ arp -a

_gateway (192.168.1.1) at a4:2b:8c:14:55:77 [ether] on eth0

device2.local (192.168.1.15) at b8:27:eb:8a:32:11 [ether] on eth0

* hostname

simply displays the current network name of your machine!

O/P

$ hostname

ubuntu-server

* ipconfig /all

Displays detailed configuration information about your network interfaces, including IP, Router, Gateway, and MAC address!

$ ip a

1: lo: <LOOPBACK, UP, LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN inet 127.0.0.1/8 scope host

2: eth0: <BROADCAST, MULTICAST, UP, LOWER_UP> mtu 1500 qdisc pfifo_fast state UP inet 192.168.10/24 brd 192.168.1.255 scope global eth0

* nbtstat -a

Helps solve problems with NetBIOS name resolution over TCP/IP!

$ nmblookup WORKGROUP
192.168.1.255 WORKGROUP<00>
192.168.1.10 WORKGROUP<00>

* netstat -r

Displays active TCP/IP connections, interface statistics, or the system's routing table.

$ netstat -r
Kernel IP routing

Destination	Gateway	Genmask	Flags	Mss	Window	Iface
default	gateway	0.0.0.0	UG	0	0	eth0
192.168.1.0	0.0.0.0	255.255.255.0	U	0	0	eth0

* nslookup

Performs DNS lookups to display IP addresses, MX records or name servers associated with specific domain!

```
$ nslookup www.google.com
Server:     8.8.8.8
Address:    8.8.8.8#53

Non-authoritative answer
Name:  www.google.com
Address: 142.250.190.36
```

* Pathping

Combines ping and traceroute to trace the route to a destination and gather statistics on packet loss at every router hop!

```
$mtr google.com                    MY traceroute [v0.92]
ubuntu-server (192.168.1.10)       Packets          pings
Host                               Loss%  Snt  Last  Avg  Best  Wst
1.-gateway                         0.0%   10   1.2   1.5  1.1   2.3
2. 10.20.30.1 (ISP gateway)        0.0%   10   12.4  13.1 11.8  15.2
3. 142.250.190.36 (google.com)     0.0%   10   22.1  22.5 21.9  23.8
```

*ping

Tests network connectivity to another host or domain using ICMP packets to see if it is reachable!

```
$ ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=14.2 ms
:
--- 8.8.8.8 ping statistics ---
3 Packets transmitted, 3 received, 0% Packet loss, time 2003
```

* ip (Network & Routing configuration)

This ip command is the modern standard for network administration in Linux. It handles IP assignment, routing and interface states.

ip <OPTIONS><OBJECTS><COMMAND>

Address Management

- Show Address

  ip address show

- Assign an IP to an interface

  ip address add 192.168.1.254/24 dev enps03

- Delete an IP from an interface:

  ip address del 192.168.1.254/24 dev enps03

Interface State Management

- Bring interface offline

  ip link set eth0 down

- Bring interface online

  ip link set eth0 up

- Enable promiscuous mode

  ip link set eth0 promisc on

Routing Management

- Add a default route

  ip route add default via 192.168.1.254 devetho

- Add a static route to a network

  ip route add 192.168.1.0/24 via 192.168.1.254

- Delete route:

  ip route delete 192.168.1.0/24 via 192.168.1.254

- Check the route for a specific IP

  ip route get 10.10.1.4

* ifconfig (Legacy Network Tool)

  once the staple command for configuring and trouble shooting networks, ifconfig has now been officially deprecated in most modern Linux distributions and replaced by the ip command suite.

* Mtr (Matt's Traceroute)

Mtr is a powerful diagnostic tool that combines the functionality of ping and traceroute.

- Basic trace and statistics

  mtr google.com

- Show numeric IP addresses only

  mtr -g google.com.

- Show both numeric IP's and hostnames

  mtr -b google.com.

- Send a specific number of pings (eg. 10).

  mtr -c 10 google.com.

* Tcdump (Packet Capture and analysis)

A robust command-line packet sniffer used to capture and analyze network traffic passing through your system (requires root/sudo privileges).

- tcdump -D
- tcdump -i eth0
- tcdump -i eth0 -c 10

- Traffic to/from a specific host
  tcdump -i eth0 host 8.8.8.8

- Traffic from a specific source (src) or destination (dst)
  tcdump -i eth0 src host 8.8.8.8
  tcdump -i eth0 dst host 8.8.8.8

- Traffic to/from an entire subnet
  tcdump -i eth0 net 10.1.0.0/24

- Capture only DNS traffic (port 53):
  tcdump -i eth0 port 53

- capture HTTPS traffic for a specific host
  tcdump -i eth0 port 443 host www.google.com

1. Ping - used to test the reachability of a host machine by sending ICMP echo Request Packets.
2. traceroute (mtr)
   Shows the detailed path and every router (hop) a packets takes to reach its final destination.
3. ip address show
   Displays the IP configuration of your machine in Linux.
4. netstat (or ss)
   Display the status of TCP and active connection
5. ip address add
   used to modify or assign a new IP address to interface.

Result:

Thus the Basic Linux commands are successfully executed and verified.

Aim:

To study different types of network cables and crimping of cable with RJ45 connector.

Types of network cables:

1) Twisted Pair Cables
- UTP (unshielded twisted Pair)
- STP (shielded twisted Pair)

2) Coaxial cable
3) optical cable RJ45 Colour code - T568B (Default order)

Pin	Colour
1	white - orange
2	orange
3	white - green
4	Blue

Pin	Colour
5	white - Blue
6	Green
7	white - Brown
8	Brown

order:

white_orange → orange → white_green → Blue → Green
white → Brown → Brown

Input:

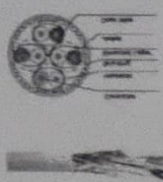
Practical-2

Aim: Study of different types of Network cables.

a) Understand different types of network cable.

Different type of cables used in networking are:

1. Unshielded Twisted Pair (UTP) Cable
2. Shielded Twisted Pair (STP) Cable
3. Coaxial Cable
4. Fibre Optic Cable

Cable type	Category	Maximum Data Transmission	Advantages/ Disadvantages	Application/Use	Image
UTP	Category 3	10 bps	**Advantages** • Cheaper in cost • Easy to install as they have a smaller overall diameter. **Disadvantages** • More prone to (EMI) Electromagnetic interference and noise	10Base-T Ethernet	
	Category 5	Up to 100 Mbps		Fast Ethernet, Gigabit Ethernet	
	Category 5e	1Gbps		Fast Ethernet, Gigabit Ethernet	
STP	Category6,6a	10Gbps	**Advantages** • Shielded. • Faster than UTP. • Less susceptible to noise and interference **Disadvantages** • Expensive • Greater installation effort	Gigabit Ethernet, 10G Ethernet (55m) Widely used in data centres	
SSTP	Category 7	10Gbps		Gigabit Ethernet, 10G Ethernet (100m)	

Input:

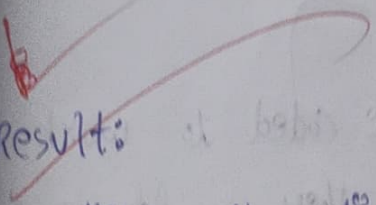
1) Cable 5e / Cable 6 / Ethernet Cable
2) RJ45 Connector
3) Crimping Cable tool
4) Cable tester

Colour sequence

WO, O	WG	B	WB	G	WBr	Br

Output: (observation)

The network cable was successfully crimped with an RJ45 connector using the T568B colour standard, and the cable connection was verified using a cable tested, as shown in the figure.



Result:

Hence the different types of network cables have studied and crimped the cable with RJ45 connector.

EXP NO: 3

Cisco Packet Tracer
(Simulation Tool)

Aim:

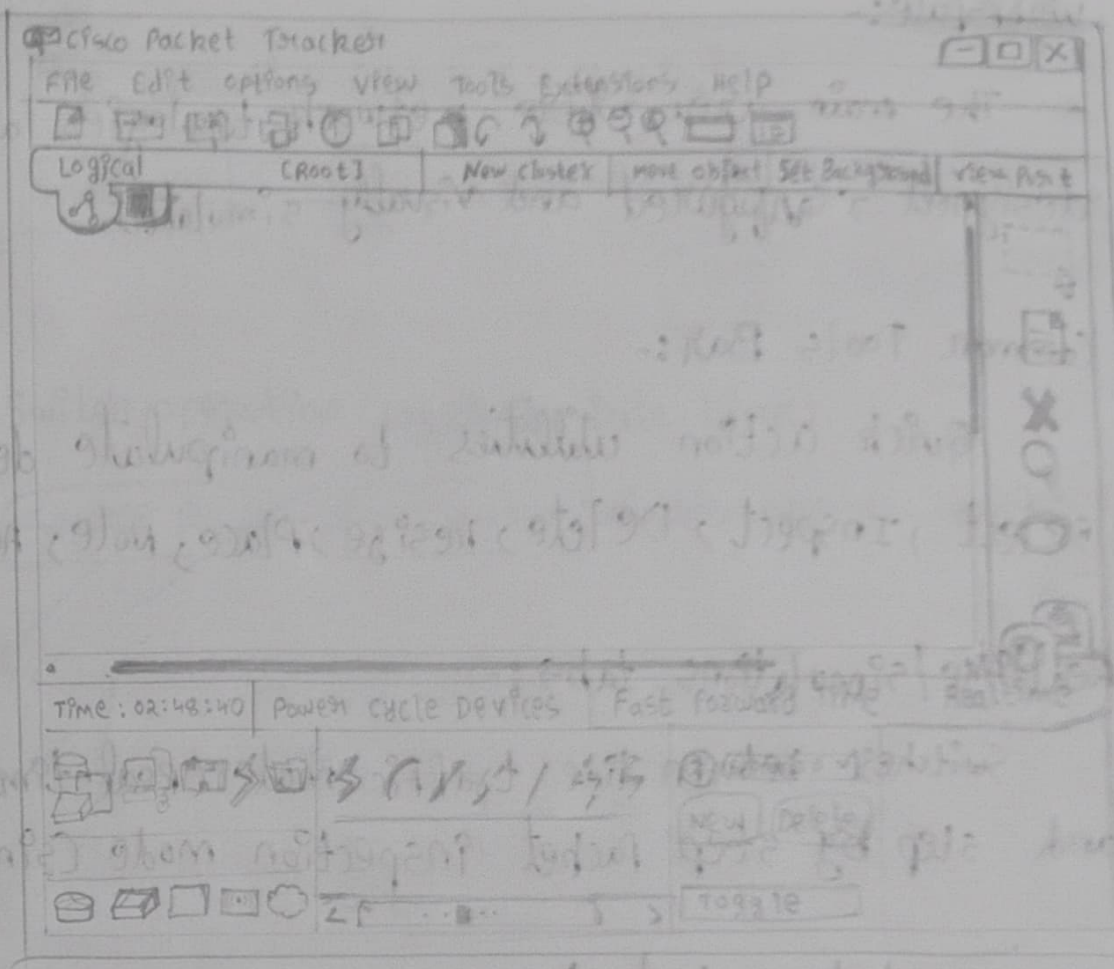
To study the packet tracer tool installation and user Interface overview.

Introduction:

A simulator, as the name suggests, simulates network devices and its environment. Packet Tracer is an exciting network design, simulation and modelling tool.

- It allows you to model complex systems without the need for dedicated equipment.
- It helps you to practice your network configuration and troubleshooting skills via computer or an Android or iOS based mobile device.
- It is available for both the Linux and windows desktop environments
- protocols in Packet Tracer are coded to work and behave in the same way as they would on real hardware.

User Interface Overview:



UI Components:

1. Menu Bar:-

Standard top menu used to manage files, e
Preferences and adjust software settings.

2. Main Toolbar:-

Provides quick access icons for common ac
like Open, Save, Zoom, Undo, Redo and Network inf

3. Logical/Physical workspace tabs:-

Toggles between the logical network topol
view and the physical geographic layout

4. Workspace:-

The main area where network topologies are designed, configured and visually simulated

5. Common Tools Bar:-

Quick action utilities to manipulate devices select, Inspect, Delete, Resize, Place, Note, Add PDU

6. Realtime/simulations tabs:-

Switches between live traffic mode (Real-time) and step by step Packet inspection mode (simulation)

7 User Created Packet box:-

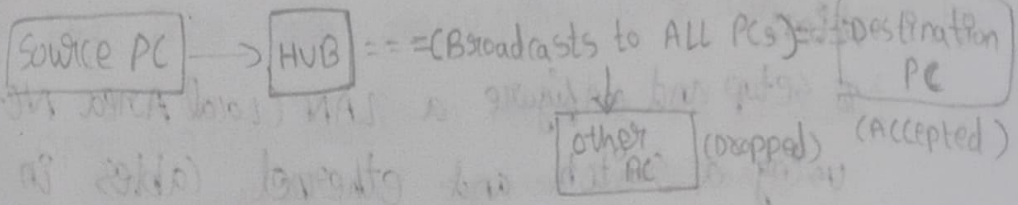
Manages Custom test Packets (PDU) and tracks their delivery success rate. Status.

Topology Configuration:-

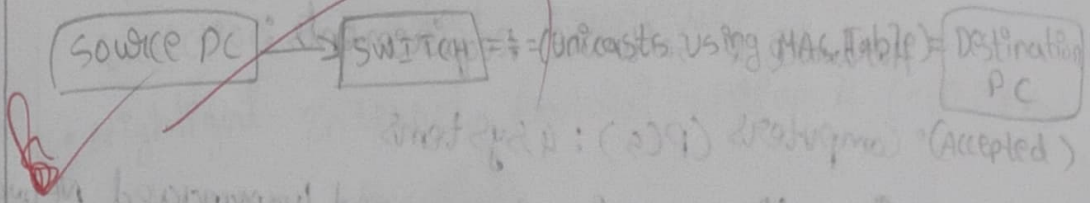
- Hub Topology:- 4 Generic PCs connected to 1 central Hub via Copper Straight through Cables.
- Switch Topology:- 4 Generic PCs connected to 1 central switch via Copper Straight through Cables.
- Addressing: All PCs in both topologies configured with unique IP addresses on the same subnet (e.g., 192.168.1.x/24).

Observations:

Hub operation (Layer 1 - Physical):



Switch operation (Layer 2 - Data Link):



Exp No: 4

Setup/configure of Local Area Network (LAN)

Aim:

To setup and configure a LAN (Local Area Network) using a switch and ethernet cables in lab.

Hardware & software Requirements:

- Computers (PCs): 4 systems
- Connecting device: 1 Managed/unmanaged Network switch (8/10/24) Ports.
- Transmission Medium: Ethernet cables (cat 5e/cat6 with RJ45 connectors).
- operating system: Windows 10/11 or Linux.

Procedure:

1. Physical Topology Design & cabling:-
   Connect host devices to the central switch.
   - Place the 4 PCs and the network switch in designated locations within the lab.
   - Plug one end of cat 5e/cat6 Ethernet cable into the ethernet port (NIC) of PC1 and the other
   - Repeat step 2 for PC2 (Port 2), PC3 (Port 3), and PC4 (Port 4)
   - verify physical link status by checking that the LED indicator lights glow green on both network adapters and switch ports.

2. IP Addressing Configuration :-

Assign Static IPv4 addresses to host computers

- Log into each computers with admin privileges
- Open control panel → Network and Internet → Network system (connections)
- Right-click on ethernet / Local Area network connection and select properties
- Highlight Internet Protocol version 4 (TCP/IPv4) and click properties
- Select "use the following IP address" and configure static IP parameters
  - PC1 : 10.1.1.1 | subnet Mask 255.0.0.0
  - PC2 : 10.1.1.2 | subnet Mask 255.0.0.0
  - PC3 : 10.1.1.3 | subnet Mask 255.0.0.0
  - PC4 : 10.1.1.4 | subnet Mask 255.0.0.0

3. Switch Management Configuration :-

Assign management IP to the switch.

- Connect an ethernet cable from PC1 to the management / console port of the switch.
- Open a web browser on PC1 and type the switch default management IP in the address bar ; enter administrative credentials.
- Navigate to Basic Network configuration and set the switch management IP to 10.1.1.5 with subnet Mask 255.0.0.0 save settings.

4. Network verification:-

Test connectivity across nodes:

- Open the Command Prompt (cmd) on PC1
- Test reachability to all network nodes by executing the following commands
  - Ping switch : Ping 10.1.1.5
  - Ping PC2 : Ping 10.1.1.2
  - Ping PC3 : Ping 10.1.1.3
  - Ping PC4 : Ping 10.1.1.4
- Ensure 0% Packet loss and stable and low latency reply times

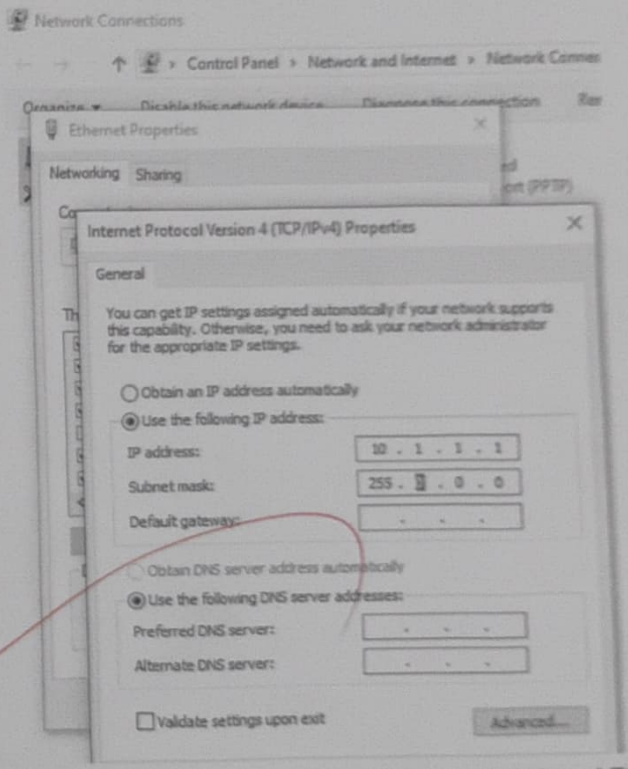
5. Resources & File sharing setup:-

Share resources over the established LAN

- On PC1, create a folder → select properties → go to the sharing tab.
- ~~Right~~ click the share, select everyone from the drop down menu; set permissions to Read/write and click share
- From PC2, press win+R, type \\10.1.1.1, and hit Enter to access and verify access LAN-Shared-Folder.

Tables:

Source Device	Target Device	Target IP	Status / Result
PC1	Network switch	10.1.1.5	Reply received (0% loss)
PC1	PC2	10.1.1.2	Reply received (0% loss)
PC1	PC3	10.1.1.3	Reply received (0% loss)
PC1	PC4	10.1.1.4	Reply received (0% loss)



Setup and configure a LAN (Local area network) using a Switch and Ethernet cables

A Local Area Network consisting of 4 PCs and 1 switch was successfully designed, physically cabled and logically configured with static class A IP address (10.1.1.0/8). Connectivity across all nodes was verified using the ping utility and network resource sharing was successfully done.

Exp. No: 5

Wireshark. (Packet sniffer tools)

Aim:

To study Packet Sniffer tools like wireshark by capturing live network traffic in promiscuous mode.

objectives:

- Understand how packet sniffing and traffic analysis work in computer networks.
- Learn distinction between promiscuous and non promiscuous network interface modes.
- understand the flow of data through TCP/IP stack layers.
- Apply display and capture filters to isolate specific network protocols.

Wireshark is an open-source network protocol analyzer that captures packets traversing a network segment in real time and displays them in a structured, human readable format.

Under Normal conditions, a Network Interface card (NIC) only process frames addressed to its specific MAC address or broadcast addresses. In promiscuous mode, the NIC bypasses this hardware address filter and passes all network recieved on the medium directly to the OS for inspection.

procedure:

1. Initial setup
- Launch wireshark as administrator
- Go to capture > options and ensure that "Enable promiscuous mode on all interfaces." is checked
- select the active network interface

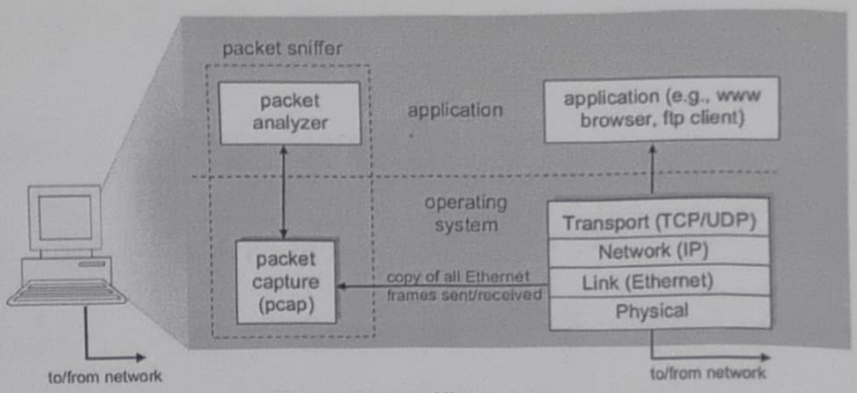


Figure 1: Packet sniffer structure

Experiments on Packet capture tool: Wireshark

4

2 Capturing DNS ... ip -

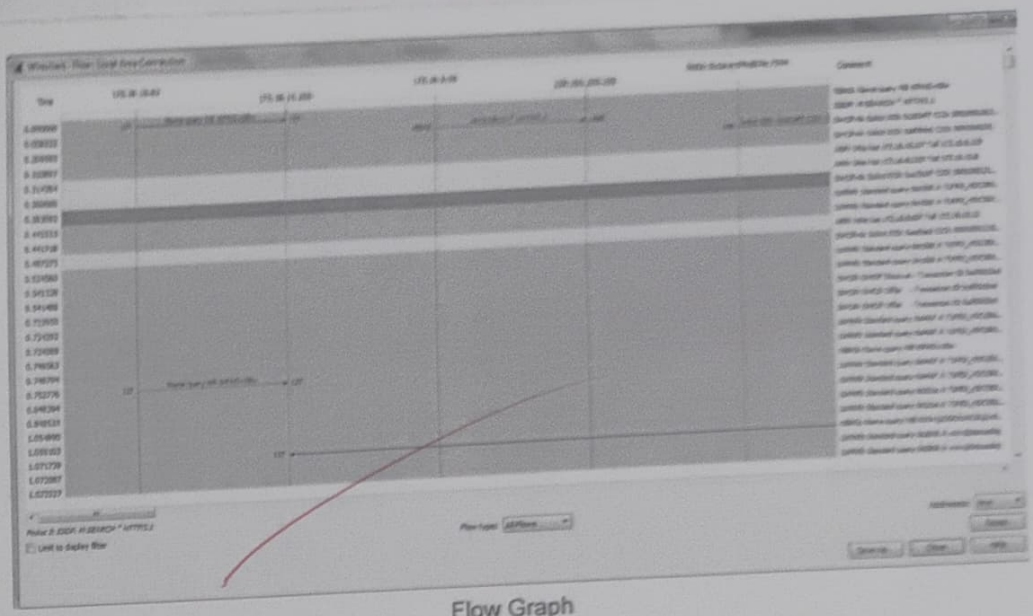
- In the terminal, clear the local DNS cache or query a site.
- Apply the display filter dns in wireshark
- Observe the DNS Query packet sent via UDP Port 53 and the corresponding DNS Answer packet.

4. Capturing HTTP/TCP traffic

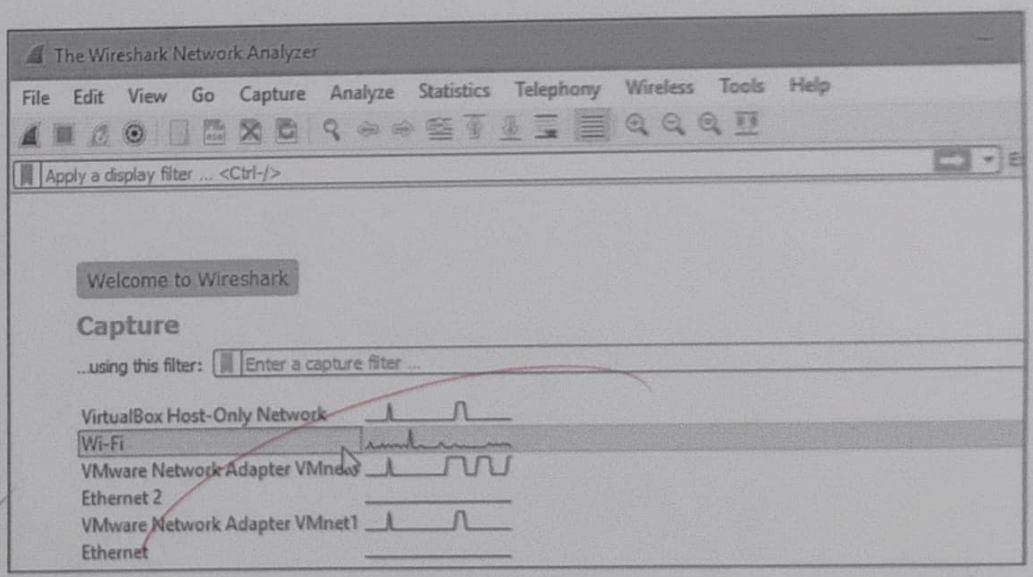
- open web browser and visit HTTP site http://neverssl.com
- Apply the display filter http or tcp.port == 80 in wireshark.
- Locate the initial TCP three way handshake (SYN, SYN-ACK, ACK) before the HTTP GET Request.

Result:

The wireshark was installed and configured to operate in promiscuous mode. Various network protocols were captured, filtered and analyzed layer-by-layer.



Flow Graph



Capturing Packets

Exp No:6

Hamming Code (Error correction)

Aim:

To write a program to implement error detection and correction using Hamming code concept. Make a test run to input data stream and verify error correction feature.

Error correction at Data Link Layer

Hamming Code is set of error-correction codes that can be used to detect and correct the errors that can occur when the data is transmitted from the sender to the receiver. It is a technique developed by R.W. Hamming for error correction.

Create sender program with below feature

1. Input to sender file should be a text of any length. Program should convert the text to binary
2. Apply Hamming Code concept on the binary data and add redundant bits to it
3. Save this output in a file called channel

C - code Implementation:

```
#include <stdio.h>
#include <string.h>

void encode (char data[], int code[]){
    code[3] = data[0] - '0';
    code[5] = data[1] - '0';
    code[6] = data[2] - '0';
    code[7] = data[3] - '0';
    code[1] = code[3] ^ code[5] ^ code[7];
    code[2] = code[3] ^ code[6] ^ code[7];
    code[4] = code[5] ^ code[6] ^ code[7];
}

void decode (int code[]){
    int P1 = code[1] ^ code[3] ^ code[5] ^ code[7];
    int P2 = code[2] ^ code[3] ^ code[6] ^ code[7];
    int P3 = code[4] ^ code[5] ^ code[6] ^ code[7];
    int err = P1*1 + P2*2 + P4*4.

if(err){
    printf("Error detected at position: %d\n", err);
    code[err] ^= 1
}else{
    printf("No error detected.\n")
}
printf("Decoded Data: %d%d%d%d\n",
        code[3], code[5], code[6], code[7]);
}
```

```
int main(){
  char data[5]; int code[8]={0};
  printf("Enter 4-bit binary data");
  scanf("%s",data);

encode(data,code);
printf("Encoded stream:");
for(int i=1;i<=7;i++) printf("%d",code[i])
Print("\n")

decode(code);
return 0;
}
```

```
Input:
  1011

Output:
  Encoded stream: 0110011
  No error detected
  Decoded Data: 1011
```

Result:

The Hamming code Algorithm for error correction and detection was successfully implemented

Exp No: 7

Flow Control

Aim:

To implement flow control at the Data Link Layer using the sliding window protocol and simulate bidirectional frame transmission with piggybacking and error handling.

Hamming code is an error correcting code designed by R.W. Hamming. It adds redundant (Parity bits at bit positions that are powers of 2. (1, 2, 4, 8) to detect and correct single bit error occuring during transmission. The error position is determined by evaluating parity equations.

The sliding window Protocol controls point-to-point data transmission between sender and receiver It allows the sender to transmitt multiple frames within specified window size- before waiting for an acknowledgment (Ack). Piggybacking attaches the acknowledgment to the outgoing data frames from the receiver, optimizing bandwidth usage.

Algorithm:

Sender Side:

1. Convert Input string message into binary ASCII format
2. Determine required redundant bits r using $2^r \geq m + r + 1$;
3. Insert Parity bits at positions 1, 2, 4, 8, ... and set remaining bits with data.
4. Calculate even parity for each redundant bit and store the encoded binary stream in channel.

Receiver Side:

1. Read frames from sender_Buffer.
2. Check frame sequence.
3. Write ACK <Expected_frame_No> to Receiver Buffer if order is correct, or NACK <Frame_No> if out-of-order or corrupt.

C code Implementation

```
#include <stdio.h>
int main(){
    int w,n,ack;
    printf("Enter window size");
    scanf("%d",&w);
    printf("Enter total number of frame:");
    scanf("%d",&n);
    int i=0
    while(i<n){
      print("\n -- sending window --\n");
      for(int j=i; j<i+w && j<n; j++){
          printf("sent frame %d\n",j);
      }
      printf("Enter last ACK received:");
      scanf("%d",&ack);
      if(ack>i){
        i=ack;
      }else{
          printf("NACK/Timeout! Resending
                  window..\n");
      }
    }
    printf("\nAll frames sent successfully!\n");
    return 0;
}
```

```
Enter window size: 3
Enter total number of frames: 5

--- Sending Window ---
Sent Frame: 0
Sent Frame 1
Sent Frame 2
Enter last ACK received: 3

--- Sending Window ---
Sent Frame 3
Sent Frame 4
Enter last ACK received: 5

All Frames sent successfully!
```

Result:

The sliding window protocol for Data Link La
Flow control was successfully implemented and simulated. The bidirectional transmission of data frames and piggybacked acknowledgments was verified.

Exp No: 8 | NMAP to discover live hosts using scans

Aim:

To discover live hosts accuracy network segments using various NMAP scanning techniques (ARP, ICMP, TCP/UDP) on Try Hackme.

ARR Scan (-PR):

uses address resolution protocol frame, extremely fast and reliable but strictly confirmed to the local subnet.

ICMP scan:

-PE : ICMP echo request
-PP : ICMP timestamp request
-PM : ICMP address mark.

•TCP ACK Ping

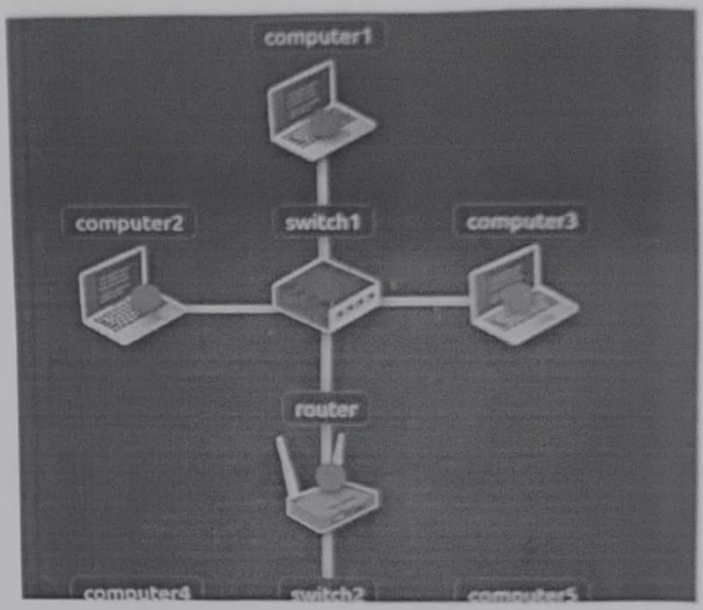
- Pa <Ports> sends a TCP (ACK) Packet directly an active target responds with RST

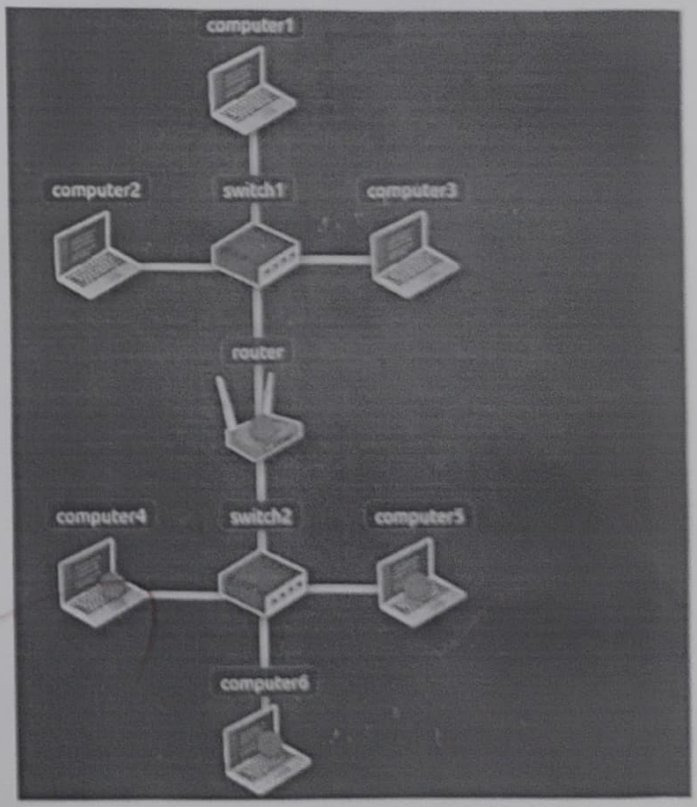
• UDP Ping:

-PU <Ports> sends UDP packet to cloud. Ports, expecting an 'ICMP type 3 code 3 error message

Procedure:

① Target specification & Preview:





Discover Live Hosts Using Nmap Scans (ARP, ICMP, TCP/UDP) on the TryHackMe

Host be up (0.00013s latency)

MAC address: 62:83:75:3A:F2:84

Nmap scan report for ip: 10.10.210.

Host is up

MAC address : 22:63:DD:18:20:CA

Nmap done : 256 IP address

Nmap [span-44] (stat - span) [span-44]

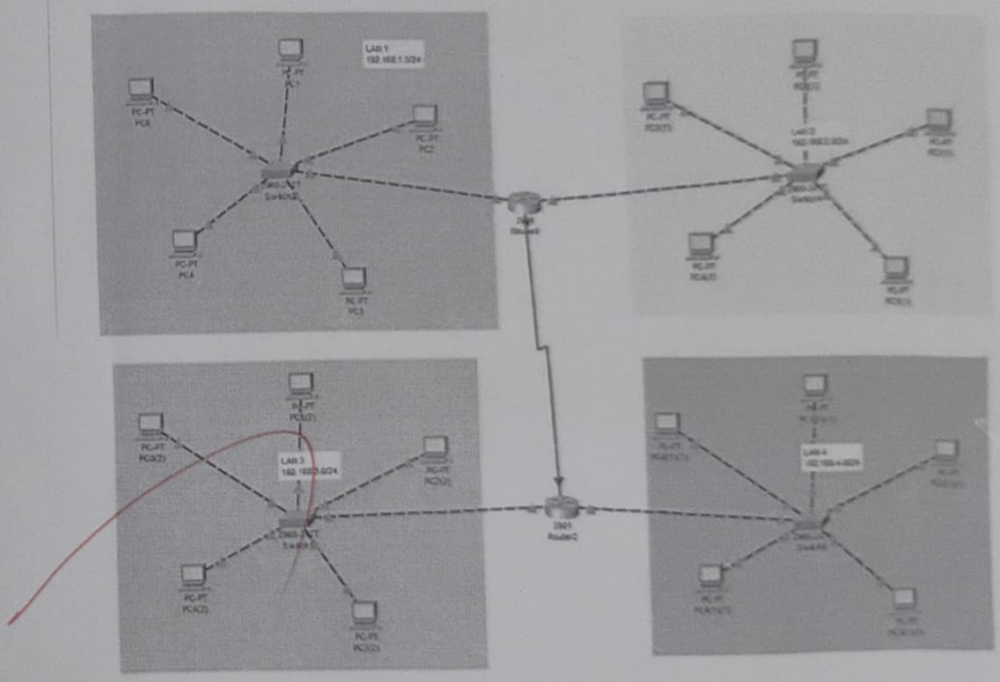
Result:

The various live hosts discovery mechanism of NMAP (ARP, ICMP, TCP & UDP ping scans) were studied and executed on Tryhackme

Exp.No:9 Implementation of subnetting using Cisco Packet Tracer

Aim:

To implement IP subnetting using the Cisco Packet tracker simulation, configure, router switches and CPs and analyze packet



Implementation of SUBNETTING in CISCO PACKET TRACER simulator

Procedure:

step ① - Topology design: In cisco packet tracer connect Pc's to their respective switch using copper cables

step ② - Router interface configuration

interface fast the net 0/1

ip address 192.168.2.1

no shutdown

exit

step ③ - switch Port configuration

enable, configure terminal, switchport mode access exit.

step ④ Host Configuration.

click each PC, assign static IP, subnet mask and default gateway in the table span

Result:

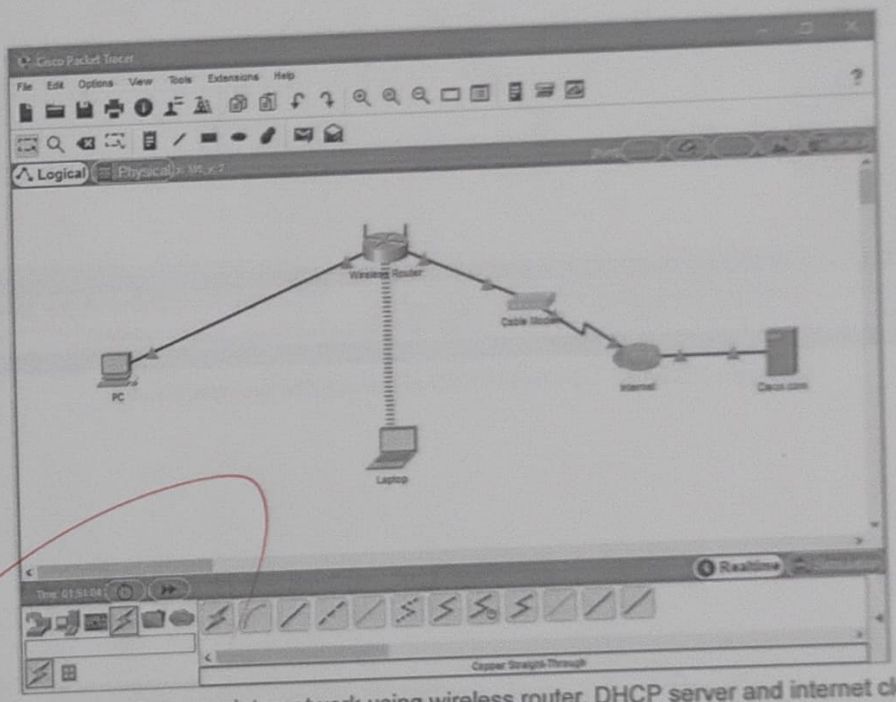
Clusters IP subnetting was implemented and tested using cisco packet tracer communication between host the subnet through a router was verified fully.

Exp. No: 10 Internetworking using Cisco Packet Tracer

Aim:

Design and configure a simple Internetwork connecting two different network connecting two differ network.

Design and configure and internetwork using



Design and configure an internetwork using wireless router, DHCP server and internet cloud

DNS services

Result:

The Internetwork topologies were designed and configured successfully executed.

Exp. No: 11

Routing at Network layer

Aim:

Simulate static routing config with primary and backup router using Cisco Packet Tracer and dynamic routing.

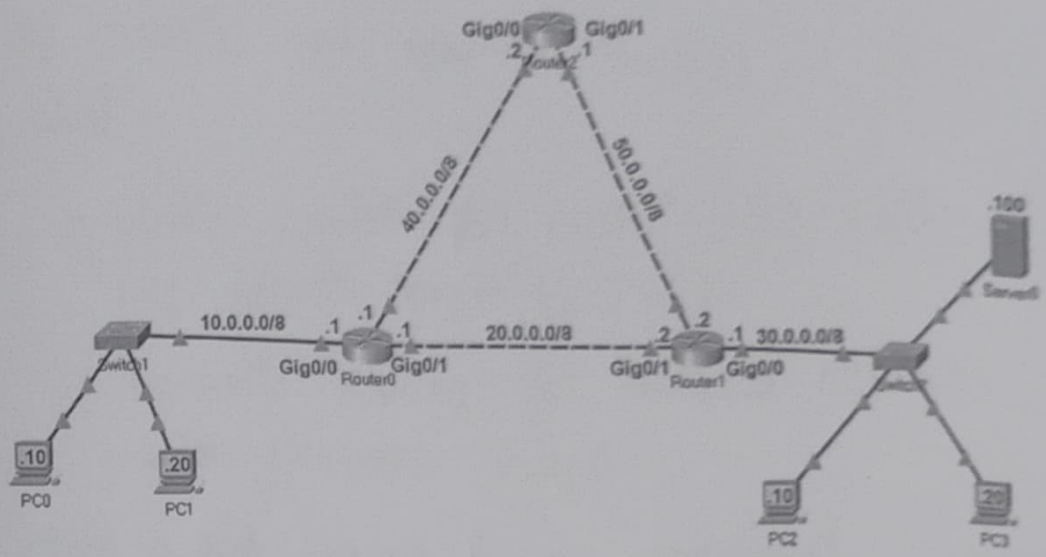
Procedure:

Static routing :-

- Place 3 routers, 2 switches Assign IP address configure IP address and bring Interfaces up
- Configure primary and backup router.
- Verification & failure.

RIP routing :-

- Build simple topography
- Configure router interfaces
- Enable RIP & advertise connected Networks.
- verification of dynamic routing.



Simulate Static Routing Configuration using CISCO Packet Tracer

```
Command Prompt

Packet Tracer PC Command Line 1.0
PC>ipconfig

FastEthernet0 Connection:(default port)
Link-local IPv6 Address.....: FE80::260:70F
IP Address.....: 20.0.0.2
Subnet Mask.....: 255.0.0.0
Default Gateway.....: 20.0.0.1

PC>ping 10.0.0.2

Pinging 10.0.0.2 with 32 bytes of data:

Request timed out.
Reply from 10.0.0.2: bytes=32 time=3ms TTL=126
Reply from 10.0.0.2: bytes=32 time=3ms TTL=126
Reply from 10.0.0.2: bytes=32 time=3ms TTL=126

Ping statistics for 10.0.0.2:
    Packets: Sent = 4, Received = 3, Lost = 1 (
Approximate round trip times in milli-seconds:
    Minimum = 3ms, Maximum = 3ms, Average = 3ms

PC>
```

Simulate RIP using CISCO Packet Tracer

Observation:

To read world coordinate points
of static and dynamic routing in cisco
packet tracer

- Static routes for predictable networks
(eg. default route to ISP)
- Dynamic routing for interconnected networks
with frequent changes

Cisco Packet Tracer is an excellent simulation
tool to visualize these concepts without
expensive hardware. Protocol behaviour

Result:

Static routing includes minimum latency
routes and are dynamic routing uses
successfully configured and verified using
Cisco Packet Tracer.

Exp.No: 12 | End to End Communication AT Transport Layer

Aim:

a]. Implementation an echo client server using TCP/UDP Sockets

b]. Implement a two way chat client ever app using Socket programming

ECHO SERVER

a] import socket

```
S = socket.socket()
S.bind (("local host", 5000))
S.listen()

c, add r = S.accept()
msg = c.recv(1024).decode()
print ("client:", msg)
c.send (msg.encode())
C.close()
B.close()
```

ECHO CLIENT

```
Import socket
S = socket.socket()
S.connect(("local host", 5000))
msg = input("Enter message")
s.send (msg: encode())
print ("Echo:", s.recv(1024).decode()).
s.close
```

Input:

Enter message: Hello server

output:

server

client connected

Received: Hello server

client

Echo From server: Hello server

b]

SERVER

```
import socket
s= socket.socket()
s.bind (("localhost", 5000))
s.listen(1)

c,a =s.accept()
while True:
    msg = c.recv(1024).decode()
    print("Client:", msg)
    if (msg="bye")
        break
    reply =input("server")
    c.send (reply.encode()).
c.close
```

CLIENT

```
import socket
s = socket.socket()
s = connect ("localhost", 5000))
while True:
    msg = input("client")
    s.send (msg.encode())
    if (msg == "bye")
        break
    print ("server:", server.recv(1024).
                                decode())
s.close
```

Input:

client : Hello
server : Hi
client : Bye

Output:

client : Hello
server : Hi

Result:

The echo client server and that chat client server applications were implemented using TCP Socket Programming API's in Python

EXP.NO:13 Implementation own Ping Program

Aim:

To Implement a ping program using Python to check the connectivity between source and destination host.

Algorithm:

- Create the destination IP address from the user.
- send the request to destination.
- wait for response.
- Calculate the response time
- Display whether the host is reachable
- Repeat the process for few request

Code:

```
import os
import time
host = input("Enter host:")
for i in range(4)
    start = time.time()
    result = os.system("ping -n 1 "+host+">null")
    end = time.time()
```

```
if result == 0:
    print("Reply from", host) "time=",
          round((end_start) * 1000, 2), "ms")

else:
    print("Request timed out")

time.sleep(1)
```

Input:

Enter host: 127.0.0.1

output:

Reply From 127.0.0.1 Time = 1.25 ms

Reply from 127.0.0.1 Time=0.98 ms

Result:

The ping program was successfully studied and implemented.

EXP.NO:14 Packet sniffing using Raw Sockets

Aim:

To write a program using RAW sockets to capture and display network packets.

Algorithm:

- Create a Raw socket
- Bind the socket to the network interface
- Receive Packets from the network
- Extract the IP header information
- Display the source IP destination IP and protocol.
- Continue capturing Packets

Code:

```
import socket
S = socket.socket(socket.AF_INET, Socket.SOCK_RAW,
                  socket.IPPROTO_IP)
s.bind(("0.0.0.0",0))
s.setsocket(socket.IPPROTO_IP, socket.IPHDRNCL))
print("Packet sniffer started...")
```

```
while True:
    data ,addr = s.recvfrom(65535)
    print("Packet received from", addr[0])
```

Input:

Network Packets

output:

Packet sniffer started...

Packet received from 192.168.1.5

Packet received from : 192.168.1.1

Packet received from : 192.120.183.14

Exp. No: 15	Analyze various types of servers using webalizer tools

Aim:

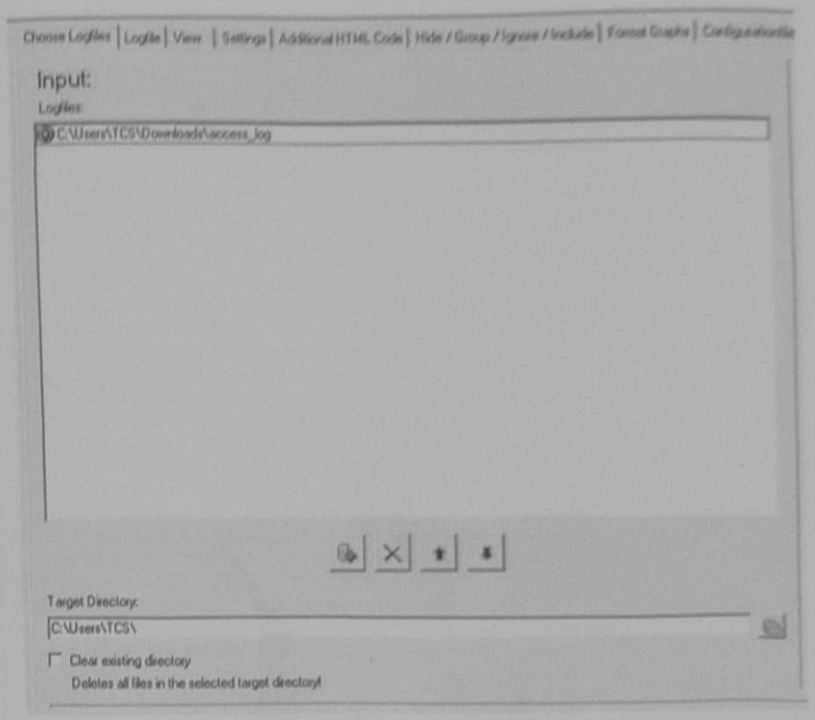
To analyze various types of well servers; access logs and generate statistics usage reports using webalizer analysis tool.

Procedure:

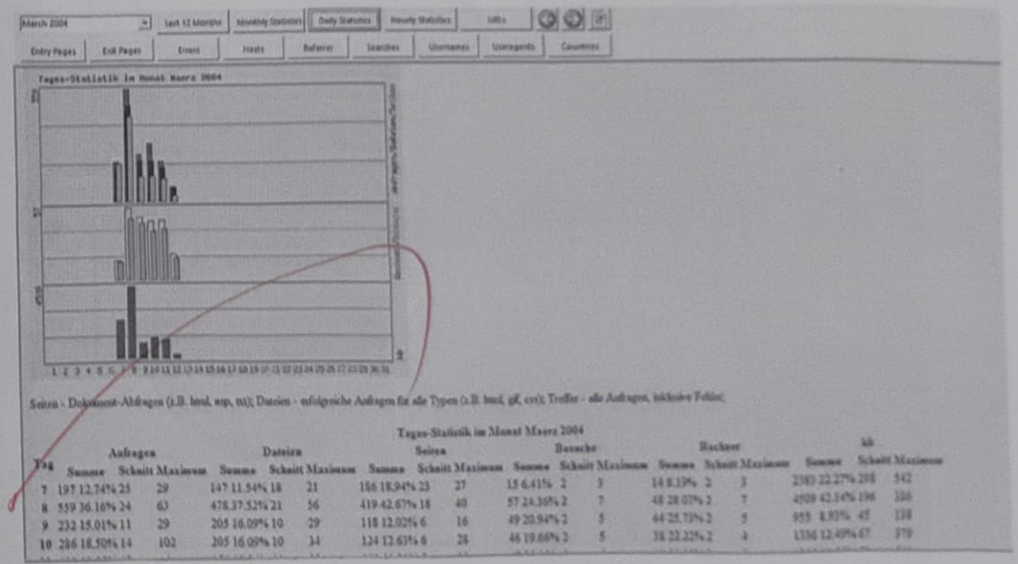
- Launch tool: run the webalizer tool app.
- Import log file: selec the input web server log.
- Configure output: specify the output target folder where generated results and HTML code
- Execute analysis: click run webalizer to parse the log records.
- view results: open the generated index.html report file in a web browser to inspect daily, host and web-agent statistics
- user agents & references client browser types operating system & URLS

Result:

The web server log file (access-log) was successfully proceeded and analyzed web server activities using the webalizer tool. comprehensive graphical and tabular reports covering hits, loss were generated and inspected



To analyze the different types of web logs using Webalizer tool.



Completed
15/10/26

webalizer tool. comprehensive graphical and tabular reports covering hits, loss were generated and inspected